

Today 11:56 AM



Lab_4_VibeCoding...

PDF



VibeCodingGitRep...

PDF

help me finish this SE LAB TASK .. tell me step by step what to do specifically the video recording part and the others too and here , use my github link too  [GitHub](https://github.com/CODESTOCKFISH/SE)
 <https://github.com/CODESTOCKFISH/SE>

Worked for 7s

Yep. I checked the lab handout **and your assigned repository**, so we can do this specifically for your Connect Four task rather than guessing.

Your assignment is **PES1UG24CS141** → **SETAPESU26/15_connect_four** , and the repository list confirms that assignment.  VibeCodingGitRepoLinks-Sec-C

The lab itself says the workflow is: inspect the assigned repo, run the Python game, record **10 seconds before changes**, use prompts to fix/add features, record **10 seconds after**, and push the finished code to your personal repository. It explicitly says **do not raise a PR to the main repo**.



Lab_4_VibeCoding_handout

I also checked your personal repo: CODESTOCKFISH/SE exists and its default branch is `main` .

What your actual task is

Your assigned game is **Connect Four vs AI**.

The starter code currently has these deliberate problems:

- `board.py` only checks **horizontal and vertical** wins.
- `game.py` has basic termination handling but needs stronger handling of invalid/full-column situations.
- `ai.py` simply picks a random legal column.
- There is no move-level success feedback.

The README requires four tasks:

1. Complete win detection — horizontal, vertical, both diagonals.
2. Complete game termination — draws, full columns, invalid moves, correct stopping after a win.
3. Improve AI — immediate winning move + block immediate player win.

4. Add concise feedback after a successful disc placement.

It also requires testing the edge cases listed in the README.

PART 1 — Before touching anything

Step 1: Clone the assigned repository

Open **PowerShell** and run:

```
</> powershell  
  
cd Desktop  
git clone https://github.com/SETAPESU26/15_connect_four.git  
cd 15_connect_four
```

Then:

```
</> powershell  
  
code .
```

That opens it in VS Code.

PART 2 — Run the original broken program

Before changing anything, run:

```
</> powershell  
  
python main.py
```

You should get a terminal Connect Four game.

The important thing is that you **actually demonstrate the broken behavior before fixing it**.

The README specifically asks you to reproduce the defect and then fix it.

What broken behavior should you demonstrate?

The easiest one is the missing diagonal detection.

You don't need to perfectly engineer a four-move diagonal in the first 10 seconds. The objective is to capture the **original/broken state**, and your final video will show the corrected behavior.

But there is a better approach

Because the assignment requires only **10 seconds before**, don't waste time playing a giant game.

Record the terminal with the original code running and make a few moves.

For example:

 Plain text

```
python main.py
```

Then play a couple of moves so the terminal clearly shows the game running.

PART 3 — EXACTLY HOW TO RECORD THE BEFORE VIDEO

This is the bit you were asking about.

The handout says:

"Record 10 seconds of video before making any changes."  Lab_4_VibeCoding_handout

The repository README makes it slightly more specific: the before video should show the **bug/broken behaviour**.

Recommended recording

Use **Windows Snipping Tool** → screen recording.

Press:

 Plain text

```
Win + Shift + R
```

Then select the terminal window.

Your 10-second BEFORE video should contain:

0–2 sec

Show:

 Plain text

```
python main.py
```

and the game starting.

2–8 sec

Make a few moves.

8–10 sec

Leave the game board visible.

Name it:

 Plain text

```
before.mp4
```

Important

Do **not** edit the code before making this recording.

The entire point is to prove:

"This is what I was given."

PART 4 — Now use the Vibe Coding tool

This is where your chat history comes in.

Your lab explicitly wants you to use prompts and learn how to get the job done in roughly **3–4 attempts**.  [Lab_4_VibeCoding_handout](#)

Don't just dump:

"Fix everything."

That's terrible evidence for a Vibe Coding lab.

Instead, use a sequence of targeted prompts.

Prompt 1 — understand the code + fix Task 1

Give the AI your current files, then use:

`</>` Plain text

I am working on the Connect Four game in this repository.

Inspect the existing main.py, game.py, board.py and ai.py before modifying anything.

Task 1:

Fix win detection in board.py.

Requirements:

- Detect horizontal four-in-a-row.
- Detect vertical four-in-a-row.
- Detect diagonal four-in-a-row in both diagonal directions.
- Do not count sequences shorter than four.
- Do not rewrite the project unnecessarily.
- Keep the existing architecture and standard-library-only setup.

Explain what is currently broken, what you changed, and why.

Then apply the changes.

Prompt 2 — game termination

Use:

`</>` Plain text

Now implement Task 2 without replacing the existing project structure.

Fix game termination so that:

- draws are detected correctly,
- full columns are handled safely,
- invalid column numbers are rejected without changing the board,
- invalid/non-numeric player input is handled,
- quitting still works,
- a winning move immediately ends the game before another turn is requested,
- the game never changes the board for an illegal move.

Keep the implementation modular and explain the changes before/after applying them.

Prompt 3 — AI

This is the important one because your original AI is literally random.

Use:

`</>` Plain text

Now implement Task 3 in ai.py.

The AI must:

1. Take an immediate winning move whenever one exists.
2. Block the player's immediate winning move whenever necessary.
3. Never select a full column.
4. Return None safely if there are no legal moves.

Keep the current Connect Four architecture.

Do not add external libraries or persistent storage.

Do not replace the entire project with an unrelated implementation.

Explain the decision-making logic and how it tests possible moves before choosing a column.

Prompt 4 — move feedback + testing

Finally:

`</>` Plain text

Implement Task 4 and verify the complete project.

Task 4:

After a successful disc placement, print concise move-level feedback exactly once per actual move.

Then review the complete implementation against the README requirements.

Create a test checklist covering:

- horizontal win
- vertical win
- diagonal win in both directions
- shorter-than-four sequence
- draw
- full column
- invalid numeric input
- invalid non-numeric input
- immediate AI win
- immediate AI block
- quitting

That gives you a very defensible chat history.

PART 5 — Test the finished game

Run:

```
</> powershell  
  
python main.py
```

Now manually test the things the README requires.

You don't necessarily need a separate screen recording of every test unless your instructor asks for it, but **you should actually test them** because the README explicitly says to.

Your checklist:

Test	Expected result
Horizontal 4	Win detected
Vertical 4	Win detected
/ diagonal 4	Win detected
\ diagonal 4	Win detected
Only 3 in a row	No win
Full column	Move rejected
Invalid number	Move rejected
Non-number input	Error handled
Draw	Game ends
AI can win	AI takes winning move
Player can win next move	AI blocks
q	Game quits

PART 6 — AFTER video

The handout requires another **10-second video after all changes**.



Lab_4_VibeCoding_handout

This one should actually show the new functionality.

Best way to record it

Run:

```
</> powershell
```

```
python main.py
```

Then record around 10 seconds.

Try to show:

1. Game starts.
2. Make a move.
3. AI responds.
4. Move feedback appears.
5. Continue until you can show the improved behavior.

Name it:

```
</> Plain text
```

```
after.mp4
```

Even better

Capture a moment where the AI **blocks your winning threat** or takes its own winning move.

That is much stronger evidence than simply showing the game running.

PART 7 — Get the final code into YOUR repo

Your professor says to push your finished work to your personal repository, not back into

SETAPESU26 .



Lab_4_VibeCoding_handout

Your destination is:

```
</> Plain text
```

```
https://github.com/CODESTOCKFISH/SE
```

First check your existing repo:

```
</> powershell

cd ..
git clone https://github.com/CODESTOCKFISH/SE.git
cd SE
```

Then create the Lab-4 folder:

```
</> powershell

mkdir Lab-4
```

Now copy the finished game into it.

Your structure should ultimately look roughly like:

```
</> Plain text

SE
|
├─ Lab-1
├─ Lab-2
├─ Lab-3
└─ Lab-4
    |
    ├─ 15_connect_four
    |   │   └─ README.md
    |   │   └─ main.py
    |   │   └─ game.py
    |   │   └─ board.py
    |   │   └─ ai.py
    |   └─ requirements.txt
    |
    └─ before.mp4
    └─ after.mp4
    └─ chat_history.pdf
```

You don't necessarily need that exact nesting unless your existing lab repo convention requires it, but keeping the game in its own folder is clean.

PART 8 — Commit and push

From your SE repo:

```
</> powershell
```

```
git status
```

Then:

```
</> powershell
```

```
git add Lab-4  
git commit -m "Complete SE Lab 4 Vibe Coding"  
git push origin main
```

Check:

```
</> powershell
```

```
git status
```

You want something like:

```
</> Plain text
```

```
nothing to commit, working tree clean
```

Then open:

<https://github.com/CODESTOCKFISH/SE> ↗

and make sure everything is actually visible.

PART 9 — Chat history PDF

This is important because the assignment explicitly asks for the **complete LLM chat history**, and the README repeats that requirement.

Don't make a tiny summary like:

"I asked ChatGPT to fix the code."

That isn't the requested evidence.

Your chat history should contain:

 Plain text

Prompt 1

↓

AI response

↓

your testing/result

↓

Prompt 2

↓

AI response

↓

your testing/result

↓

Prompt 3

↓

AI response

↓

your testing/result

↓

Prompt 4

↓

Export the conversation as PDF.

Call it:

 Plain text

chat_history.pdf

PART 10 — Final submission

Your Lab-4 folder should contain the three required categories from the assignment:

1. Videos

 Plain text

before.mp4

after.mp4

The lab specifically requires before-and-after videos.



Lab_4_VibeCoding_handout

2. Updated Code

</> Plain text

```
15_connect_four/  
  main.py  
  game.py  
  board.py  
  ai.py  
  requirements.txt
```

3. Chat history

</> Plain text

```
chat_history.pdf
```

Then submit the assignment through your LMS.

One thing I would NOT do

Don't modify `SETAPESU26/15_connect_four` directly.

Your assignment says to clone/fork the assigned repo, make changes, push to your personal account, and **not create a PR to the main repo**.  [Lab_4_VibeCoding_handout](#)

You already have the correct personal repo:

CODESTOCKFISH/SE

So use that as the submission destination.

Your exact workflow from here

</> Plain text

```
1. Clone 15_connect_four  
  ↓  
2. Run original game  
  ↓  
3. Record BEFORE.mp4  
  ↓  
4. Open code in VS Code  
  ↓  
5. Prompt 1 → fix win detection
```

- ↓
6. Prompt 2 → fix termination
- ↓
7. Prompt 3 → improve AI
- ↓
8. Prompt 4 → move feedback + testing
- ↓
9. Test everything
- ↓
10. Record AFTER.mp4
- ↓

I also inspected the actual starter `board.py` and `ai.py`, so we know this isn't theoretical: **the diagonal detection is genuinely missing, and the AI is currently random**, which gives you very clear before/after evidence.

 Lab_4_VibeCoding_handout

Next, the most useful move is to **clone `15_connect_four`, run it, and send me the terminal output + the contents of the four Python files after cloning**. Then I can walk you through the actual Vibe Coding prompts and testing one step at a time rather than making you figure out the implementation yourself.

can u make these 4 prompts into 2 prompts which i shall communicate to you itself because u already have memory of the whole project , just to stay within the 4 prompt limit

Yes. In fact, **2 strong prompts are better than 4 weak ones** here. The assignment says "under three to four attempts," so 2 substantive coding prompts keeps you comfortably inside the limit.

Since I already have the repository structure and the original code we inspected, use these **exactly**.

Prompt 1 — Win detection + game termination

 Plain text

I am working on the Connect Four project you have already inspected. Modify the existing code, especiall

Complete the following two tasks from the README:

TASK 1 – Complete win detection:

- Detect horizontal four-in-a-row.
- Detect vertical four-in-a-row.
- Detect both diagonal directions.
- A sequence shorter than four must not count as a win.
- Preserve the existing Board architecture.

TASK 2 – Complete game termination:

- Correctly handle draws.
- Correctly handle full columns.
- Reject invalid column numbers without changing the board.
- Handle invalid/non-numeric player input safely.
- Preserve quitting with 'q'.
- A winning move must immediately terminate the game before another turn is requested.
- No illegal move should modify the board.

Then test the code.

Prompt 2 — AI + feedback + complete verification

`</>` Plain text

Continue working on the same Connect Four project and the changes we just made. Do not rewrite the proje

Complete the remaining README requirements:

TASK 3 – Improve the AI:

- The AI must take an immediate winning move whenever one exists.
- The AI must block the player's immediate winning move when necessary.
- The AI must never select a full/illegal column.
- If no legal columns exist, handle this safely.

TASK 4 – Move-level feedback:

- After a successful disc placement, provide concise feedback.
- Feedback must occur exactly once per actual move.
- It must not be printed repeatedly while winner detection or AI analysis examines the board.

Then perform a complete review of the finished implementation against the README requirements and identi

Verify the following cases:

1. Horizontal win

~ ~ ~ ~ ~

Your prompt count

You'll have:

Attempt 1: Prompt 1 → Tasks 1 & 2

Attempt 2: Prompt 2 → Tasks 3 & 4 + verification

That's it.

And importantly, **don't send me a prompt like "fix this" before these** if you're trying to preserve the assignment's attempt count. Start your Vibe Coding chat with Prompt 1.

The README specifically asks you to keep the complete LLM history, test the generated code against the stated behavior, and avoid replacing the project with an unrelated implementation, so these prompts deliberately make those requirements explicit.  Lab_4_VibeCoding_handout

One correction to my previous instructions: you don't need to send me the Python files again. I already inspected the assigned repository's `main.py` , `game.py` , `board.py` , `ai.py` , and `README`, so we can work from that exact starting point.

I am working on the Connect Four project you have already inspected. Modify the existing code, especially `board.py` and `game.py`, without replacing the project with an unrelated implementation.

Complete the following two tasks from the README:

TASK 1 — Complete win detection:

- Detect horizontal four-in-a-row.
- Detect vertical four-in-a-row.
- Detect both diagonal directions.
- A sequence shorter than four must not count as a win.
- Preserve the existing Board architecture.

TASK 2 — Complete game termination:

- Correctly handle draws.
- Correctly handle full columns.
- Reject invalid column numbers without changing the board.
- Handle invalid/non-numeric player input safely.
- Preserve quitting with 'q'.
- A winning move must immediately terminate the game before another turn is requested.
- No illegal move should modify the board.

Do not add external dependencies or persistent storage.
Keep the existing modular structure and make only the changes necessary.

Before modifying the files, briefly explain what is currently broken in the existing implementation and then implement the fixes.
Afterward, summarize exactly what changed and how I can test it.

What is currently broken

From the starter code I inspected:

- `board.py` checks only **horizontal** $(0,1)$ and **vertical** $(1,0)$ directions. Both diagonal directions are missing.
- The existing `drop()` already prevents out-of-range columns and full-column placement, but `game.py` needs more consistent handling of those cases.
- `game.py` already stops immediately after `winner()` and `full()` checks, so that part is structurally correct; we'll preserve it while tightening invalid-move handling.
- The existing architecture stays intact: `Game` → `Board/AI`, with no external packages or persistence.

Changes for Attempt 1

Use this implementation for `board.py` :

`</>` Python

```
ROWS, COLS = 6, 7
```

```
class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        if not isinstance(col, int) or not 0 <= col < COLS:
            return None

        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r

        return None

    def full(self):
        return all(self.grid[0][c] != "." for c in range(COLS))
```

The important change is:

`</>` Python

```
directions = [
    (0, 1),
    (1, 0),
    (1, 1),
    (1, -1),
]
```

So the four directions are now:

</> Plain text

```
→ horizontal
↓ vertical
↘ diagonal
✓ diagonal
```

And because it still checks **four positions**, three-in-a-row won't accidentally count as a win.

For `game.py`, use:

</> Python

```
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")

        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()

                if raw == "q":
```

What this accomplishes

- 1–7 accepted.
- 0 , 8 , -1 , etc. rejected.
- "hello" rejected without crashing.
- q still quits.
- Full columns don't modify the board.
- A winning move ends the game **before the next turn**.
- Draws terminate the game.
- The board architecture is unchanged.
- No dependencies or persistence added.

- The move feedback is already included here, but that's technically **Task 4**, so don't treat that as your second prompt's AI work; we'll still have Prompt 2 handle/verify the final feedback requirement.

Important for your lab evidence

If you **haven't recorded the BEFORE video yet, stop here and don't modify your local files yet**. Your assignment explicitly requires the 10-second recording before changes.  Lab_4_VibeCoding_handout

If you've already recorded it, apply these changes, run:

```
</> powershell
```

```
python main.py
```

and test at minimum:

```
</> Plain text
```

```
horizontal  
vertical  
diagonal \  
diagonal /  
invalid input  
full column  
q
```

Then we'll use **Prompt 2** for the AI + final verification.

Continue working on the same Connect Four project and the changes we just made. Do not rewrite the project or introduce unnecessary dependencies.

Complete the remaining README requirements:

TASK 3 — Improve the AI:

- The AI must take an immediate winning move whenever one exists.
- The AI must block the player's immediate winning move when necessary.
- The AI must never select a full/illegal column.
- If no legal columns exist, handle this safely.

TASK 4 — Move-level feedback:

- After a successful disc placement, provide concise feedback.
- Feedback must occur exactly once per actual move.
- It must not be printed repeatedly while winner detection or AI analysis examines the board.

Then perform a complete review of the finished implementation against the README requirements and identify/fix any remaining issues.

Verify the following cases:

1. Horizontal win
2. Vertical win
3. Both diagonal wins
4. Three-in-a-row not incorrectly counted as a win
5. Draw
6. Full column
7. Invalid numeric input
8. Non-numeric input
9. Immediate AI winning move
10. Immediate AI block
11. Quitting with q

Keep all state in memory, use the existing architecture, and do not add CSV, JSON, SQLite, or other persistence.

Finally, give me a concise summary of every change made and the exact tests I should demonstrate in my final video.

Done. For **Attempt 2**, the remaining work is `ai.py` plus the final cleanup/review of `game.py`. I'm keeping the exact architecture of the starter project and not adding any dependencies or persistence.

1. Replace `ai.py` with this

`</>` Python

```
class AI:
    def choose_column(self, board, me="O", opponent="X"):
        legal = [
            c for c in range(7)
            if board.grid[0][c] == "."
        ]

        if not legal:
```

```

        return None

    # 1. Take an immediate winning move if possible.
    for col in legal:
        row = board.drop(col, me)

        if row is not None:
            winning = board.winner(me)
            board.grid[row][col] = "."

            if winning:
                return col

```

What this does

The AI now follows this priority:

</> Plain text

Can I win immediately?

↓ YES

WIN

↓ NO

Can the player win immediately?

↓ YES

BLOCK

↓ NO

Choose any legal column

And importantly, the temporary move is removed immediately after checking:

</> Python

```
board.grid[row][col] = "."
```

So the AI's analysis **doesn't actually alter the board**.

2. Final `game.py`

I would make one small cleanup to the `game.py` from Attempt 1 so that the AI's `None` result is treated as a draw rather than as an invalid move.

Use:

 Python

```
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")

        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()

                if raw == "q":
```

3. Why the move feedback is correct

This line:

 Python

```
print(f"{self.turn} placed a disc in column {col + 1}.")
```

is after:

 Python

```
row = self.board.drop(col, self.turn)
```

and only runs if:

 Python

```
row is not None
```

Therefore:

Successful move

</> Plain text

X placed a disc in column 4.

Exactly once.

Invalid move

</> Plain text

Column unavailable. Choose another column.

No fake "X placed..." message.

AI analysis

The AI temporarily places a disc, calls `board.winner()` , and removes it.

It does **not** call the feedback code in `game.py` .

Therefore AI analysis produces **zero feedback messages**.

That's exactly what Task 4 wants.

4. Complete requirement review

Against the README, your final implementation now covers:

Requirement	Status
Horizontal win	✓
Vertical win	✓
Diagonal ↘ win	✓
Diagonal ↗ win	✓
Less than 4 doesn't win	✓

Requirement	Status
Draw detection	✓
Full column handling	✓
Invalid numeric input	✓
Non-numeric input	✓
q quitting	✓
Winning move immediately ends game	✓
AI immediate winning move	✓
AI blocks immediate player win	✓
AI never chooses full column	✓
AI handles no legal moves	✓
Move feedback once per move	✓
No external dependencies	✓
No persistent storage	✓
Existing modular architecture preserved	✓

This also aligns with the assignment's requirement to test the horizontal, vertical, diagonal, draw, full-column, invalid-input, AI-win, AI-block, and quitting cases.

 Lab_4_VibeCoding_handout

5. Now actually test it

Run:

```
</> powershell
```

```
python main.py
```

Test 1 — Invalid text

Enter:

</> Plain text

```
hello
```

Expected:

</> Plain text

```
Invalid input. Enter a column number from 1 to 7, or q.
```

Game continues.

Test 2 — Invalid numbers

Try:

</> Plain text

```
0
```

and:

</> Plain text

```
8
```

Expected:

</> Plain text

```
Invalid column. Choose a number from 1 to 7.
```

The board should remain unchanged.

Test 3 — Full column

Fill one column completely.

Then try that same column.

Expected:

</> Plain text

Column unavailable. Choose another column.

No disc should be added.

Test 4 — Horizontal

Create:

</> Plain text

X X X X

Expected:

</> Plain text

X wins!

And importantly, the game should end immediately.

Test 5 — Vertical

Create:

</> Plain text

X
X
X
X

Expected:

</> Plain text

X wins!

Test 6 — Diagonal

Test both:

</> Plain text

```
X . . .  
. X . .  
. . X .  
. . . X
```

and:

</> Plain text

```
. . . X  
. . X .  
. X . .  
X . . .
```

Both must produce a win.

Test 7 — Three in a row

Create:

</> Plain text

```
X X X
```

without a fourth X.

Expected:

No win.

This specifically checks that we didn't accidentally change the condition from four to three.

Test 8 — AI winning move

Set up a situation where AI has:

</> Plain text

0 0 0 .

with that fourth position playable.

The AI should select that column rather than making a random move.

Test 9 — AI block

Set up:

</> Plain text

X X X .

where the . is a legal playable position.

The AI should select that column to block you.

Test 10 — Quit

Enter:

</> Plain text

q

Expected:

</> Plain text

Game ended.

6. What to show in your AFTER video

Don't try to cram all 11 tests into 10 seconds. 🤪

The video only needs to demonstrate that the **finished game works**.

I'd record this:

Seconds 0–2

Start:

 Plain text

```
python main.py
```

Seconds 2–5

Make a move and show:

 Plain text

```
X placed a disc in column X.
```

Then show the AI responding.

Seconds 5–10

If possible, demonstrate the **AI blocking an immediate winning threat** or taking an immediate winning move.

That gives your professor actual evidence of the new AI functionality rather than merely showing that the program launches.

The assignment specifically asks for the after video to show the bug fixed and new features working.

 Lab_4_VibeCoding_handout

Your 2-attempt chat is now complete

Your LLM history should effectively show:

Attempt 1

Fix Tasks 1 & 2.

↓

Attempt 2

Fix Tasks 3 & 4 and verify everything.

That's a very clean demonstration of the "get the job done in under three to four attempts" objective.

 Lab_4_VibeCoding_handout

Final Lab-4 folder

</> Plain text

```
Lab-4/
|
├─ before.mp4
├─ after.mp4
├─ chat_history.pdf
|
└─ 15_connect_four/
    ├── README.md
    ├── main.py
    ├── game.py
    ├── board.py
    ├── ai.py
    └─ requirements.txt
```

Then push that whole Lab-4 folder into:

CODESTOCKFISH/SE

The assignment's stated deliverables are exactly the before/after videos, updated code, and exported chat history.  [Lab_4_VibeCoding_handout](#)

One caveat: I can verify the implementation logically from the code we inspected, but I haven't executed your local Python game here. So don't claim in the chat history that all 11 tests passed until you've actually run them. That's the one bit you need to do yourself.